

TreeFS

The Complete Guide

A Persistent Unix-like Virtual File System in C — Explained From First Principles

A ground-up tutorial covering data structures, C memory management, recursion, path resolution, and persistence — written so that a student with basic-to-intermediate C knowledge can read the source code afterward and understand why every major function exists.

TreeFS version 1.0
Companion document to TreeFS.zip

Table of Contents

Table of Contents	2
1. What Are We Building?	4
2. What Is a File System?	4
3. Why Is a Tree the Right Data Structure?	5
4. N-ary Trees	6
5. C Structs and Pointers	7
6. The Node Structure	8
7. Creating Nodes	9
8. Adding Children	10
9. Searching the Tree	11
10. Recursion	11
11. Paths	12
12. mkdir	13
13. touch	14
14. ls	15
15. cd	15
16. pwd	15
17. tree	16
18. rm	17
19. File Contents	18
20. Persistence	19
21. The Serialization Format	19
22. Deserialization	20
23. Command Parsing	21
24. Modular Architecture	22
25. Memory Management	22
26. Testing: What Was Actually Run	23
27. Complexity Analysis	25
28. Design Decisions and Alternatives Considered	26

29. What a Real File System Does Differently	27
30. Realistic Future Extensions	27
31. Complete Code Walkthrough: How the Files Connect	28

1. What Are We Building?

TreeFS is a small program that behaves like a miniature Unix filesystem. You run it, you get a prompt (`treefs:/ $`), and you can type familiar commands — `mkdir`, `cd`, `touch`, `ls`, `cat` — to build a tree of directories and files. Unlike the real filesystem your operating system gives you, TreeFS's tree lives entirely inside the memory of one running process. When the process ends, that memory would normally vanish — so TreeFS also knows how to write its tree out to an ordinary file on your real disk, and read it back in next time, so your directories and files are still there the next time you run the program.

Data flows through five layers, each one only talking to the layer directly below it:



Figure 1.1 — how a keystroke eventually becomes a byte on disk.

What TreeFS is not

It is worth being precise about this up front, because the word “filesystem” carries a lot of baggage. TreeFS is **not** a kernel module, a device driver, a FUSE (Filesystem in Userspace) mount, or anything the rest of your operating system can see as a real, browsable drive. You cannot `cd` into it from a normal terminal, and other programs cannot open its files. It is a self-contained simulation: a tree data structure with a command-line front end and a save file. That is a deliberate scope decision, not an accident — Section 29 explains exactly what a real filesystem does that TreeFS intentionally leaves out, and why.

2. What Is a File System?

Strip away the disk hardware and the operating system kernel, and a filesystem is really just an answer to one question: *given a name, how do I find the thing it refers to?* Everything else — directories, paths, permissions — exists to make that lookup organized instead of chaotic.

Files and directories

A **file** is a named container for data — a sequence of bytes with no structure the filesystem itself understands. A **directory** is a named container for more entries: other files, or other directories. This is a recursive definition (directories contain things that can themselves be directories), and recursive definitions are exactly what tree data structures are built to represent, which is the entire subject of Section 3.

Paths and the current working directory

Because directories can nest, a single name like `tree.c` is not enough to uniquely identify a file — there could be a `tree.c` inside `/projects/dsa` and a completely different one inside `/backup/old`. A **path** solves this by spelling out the full chain of directory names from some starting point down to the target.

An **absolute path** (like `/projects/dsa/tree.c`) starts from the root and is unambiguous no matter where you currently “are” in the tree. A **relative path** (like `dsa/tree.c`) is interpreted starting from the shell's notion of “where you currently are” — its **current working directory**, often abbreviated **cwd**. Every interactive filesystem shell, including TreeFS, keeps track of exactly one cwd at a time, and updates it every time you run `cd`.

Two special names appear inside paths everywhere: `.` means “stay right here”, and `..` means “go up one level, to my parent directory.” Both are handled by TreeFS's path resolver, covered in full in Section 11.

3. Why Is a Tree the Right Data Structure?

What is a tree, informally?

A tree is a collection of **nodes** connected by **edges**, with one special node called the **root** that has no parent, and every other node having exactly one parent. A node with no children is called a **leaf**. The nodes reachable from some node *N*, including *N* itself, are called the **subtree** rooted at *N*. The **depth** of a node is the number of edges on the path from the root down to it.

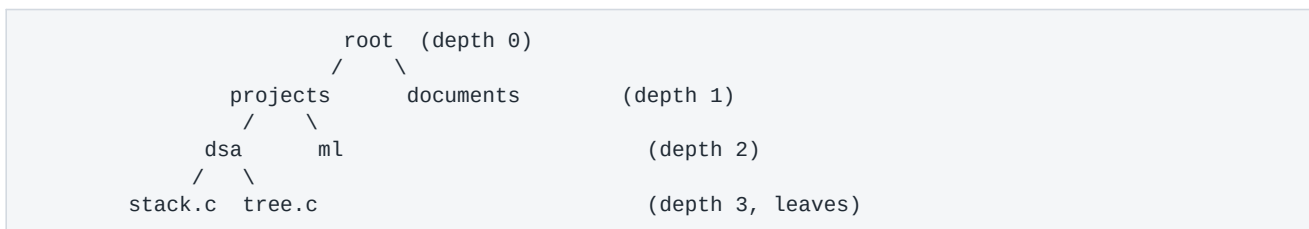


Figure 3.1 — the vocabulary of trees, mapped onto a small filesystem.

Mapping filesystem concepts onto tree concepts

Filesystem idea	Tree idea
A directory	An internal node (it can have children)
A file	A leaf node (it never has children)
The filesystem root, "/"	The tree's root node
A path like /a/b/c	The sequence of edges from the root to node c
Listing a directory's contents	Visiting a node's children
Deleting an entire directory tree	Deleting a subtree

This mapping is not a loose analogy; it is the actual design of TreeFS. The `Node` struct defined in Section 6 is both “a directory” and “a tree node” at the same time — there is no separate translation layer between the two ideas.

4. N-ary Trees

A binary tree limits every node to at most two children. That works beautifully for sorted data (binary search trees) but is the wrong shape for a filesystem: a directory might reasonably contain 0, 1, or 200 entries, decided entirely by the user at runtime, with no natural way to fit that into “left child” and “right child.” What we need is an **N-ary tree**: a tree where a node can have any number of children.

The child/sibling representation

The most common way to implement an N-ary tree without a fixed limit is to give every node exactly two pointers — not “left/right”, but **first child** and **next sibling**. A node's children are not stored as a list-of-pointers field; instead, the node points to its *first* child, and each child points to the next one in line, forming a singly linked list strung between siblings.

```

projects
|
| children
v
dsa ---next--> ml ---next--> README.txt ---next--> NULL

```

Figure 4.1 — “projects” has three children, linked to each other via `next`, not to “projects” directly (except the first one).

Walking every child of `projects` means: look at `projects->children` to get the first one (`dsa`), then repeatedly follow `->next` until you hit `NULL`. This is precisely what `fs_ls()` and `print_tree()` do (Sections 14 and 17).

Why not a fixed-size array of children?

An alternative design gives every node a field like `Node children[64];`. This is simpler to read at a glance, but it has two real costs. First, it wastes memory: a directory with one file still reserves space for 64 pointers. Second, and worse, it imposes an arbitrary ceiling — the 65th `mkdir` in the same directory would have nowhere to go. The child/sibling representation uses exactly two pointers per node no matter how many children it ends up with, and never runs out of room. The trade-off is that finding the *N*th child, or checking whether a specific name already exists, takes $O(\text{children})$ time (a linked-list walk) instead of $O(1)$ array indexing — a trade TreeFS makes deliberately, since directories a human types into interactively are never going to be large enough for that to matter. Section 27 makes this trade-off precise with real complexity bounds.

5. C Structs and Pointers

Before reading the actual TreeFS code, it helps to have these ideas solid, since every later section leans on them.

Structs

A `struct` bundles several named, possibly different-typed values into one unit. Think of it as a small, custom record type: a `Node` struct bundles together a name, a type, some content, and three pointers, and C treats the whole bundle as one value you can pass around, copy, or take the address of.

Pointers and the stack vs. the heap

A pointer is a variable whose value is a memory address — “where to find something” rather than the something itself. Local variables (an `int x;` inside a function) live on the **stack**, a region of memory that is automatically reclaimed the instant the function returns. That automatic cleanup is convenient, but it also means a pointer to a stack variable becomes meaningless once that function returns — the memory it pointed to may already have been reused.

The **heap** is different: memory you explicitly request with `malloc()` or `calloc()` stays valid until you explicitly give it back with `free()`, no matter which function allocated it or which function later uses it. This is exactly what a filesystem tree needs: a directory created inside one function call (handling the `mkdir` command) must still exist long after that function has returned, possibly for the entire remaining lifetime of the program. Every `Node` in TreeFS therefore lives on the heap.

Self-referential structs

A struct is allowed to contain a pointer to its own type — not the struct itself (which would be an infinitely large, impossible definition), but a *pointer* to one, which is just an address and therefore has a fixed, small size regardless of what it points to. This is exactly how `Node` links to other `Nodes` (its parent, its first child, its next sibling), and it is what makes any linked data structure — lists, trees, graphs — possible in C.

The allocation functions used in TreeFS

Function	What it does	Used where in TreeFS
<code>calloc(1, size)</code>	Allocates memory and zero-fills it	<code>create_node()</code> — guarantees a fresh node's pointers start as NULL
<code>malloc(size)</code>	Allocates memory, leaves it uninitialized	<code>fs_write()</code> — immediately overwritten with real content, so zeroing would be wasted work
<code>free(ptr)</code>	Returns memory to the allocator	<code>free_tree()</code> (nodes), <code>fs_write()/rename</code> (old content), <code>fs_destroy()</code>

TreeFS never calls `realloc()` on a Node itself — a node's identity (its address) must stay fixed for as long as anything else points to it, and `realloc()` is explicitly allowed to move memory to a new address. File content, by contrast, is replaced wholesale on every write (old buffer freed, new one allocated) rather than resized in place, which keeps the ownership story for content simple: exactly one pointer, in exactly one node, owns it at any moment.

6. The Node Structure

```
typedef enum {
    FILE_NODE,
    DIRECTORY_NODE
} NodeType;

typedef struct Node {
    char name[MAX_NAME_LEN + 1]; /* NUL-terminated component name */
    NodeType type;
    char *content;                /* heap content; NULL for directories */

    struct Node *parent;          /* NULL only for the root */
    struct Node *children;        /* first child, or NULL */
    struct Node *next;           /* next sibling, or NULL */
} Node;
```

src/tree.h — the complete Node definition.

Field	Stores	Owned by	Changes when...
name	This node's own component name, e.g. "tree.c" (never a full path)	The node itself (it's a fixed-size array embedded in the struct, not a pointer)	rename is called
type	FILE_NODE or DIRECTORY_NODE	The node	Never — fixed at creation
content	A file's data as a heap-allocated, NUL-terminated string; always NULL for directories	The node (freed when the node is freed or overwritten)	write is called
parent	A pointer back up to the containing directory; NULL only for the root	Not owned — a non-owning back-reference	add_child / remove_child

Field	Stores	Owned by	Changes when...
children	A pointer to this node's first child, or NULL if it has none	Not owned by this field — the children are owned collectively by the tree	add_child / remove_child on a child of this node
next	A pointer to the next sibling in the same parent's child list, or NULL if last	Not owned	add_child / remove_child

Only two of these six fields represent genuine *ownership* of heap memory that must eventually be freed: content, and the node itself. parent, children, and next are all just links into a structure that is owned collectively, by the tree as a whole, starting from the root — which is exactly why `free_tree()` (Section 18) only ever needs to be called once, on the root, to release everything.

`MAX_NAME_LEN` is 127, making name a 128-byte array (127 characters plus the terminating `'\0'`). This bound exists so that names have a fixed, predictable size known at compile time, and so `is_valid_name()` (Section 7) has a concrete limit to enforce rather than an unbounded one that could be abused to exhaust memory with a single absurdly long command.

7. Creating Nodes

```
Node *create_node(const char *name, NodeType type)
{
    if (!is_valid_name(name)) {
        return NULL;
    }

    Node *node = calloc(1, sizeof(Node));
    if (node == NULL) {
        fprintf(stderr, "treefs: error: out of memory while creating node\n");
        return NULL;
    }

    strncpy(node->name, name, MAX_NAME_LEN);
    node->name[MAX_NAME_LEN] = '\0';

    node->type = type;
    node->content = NULL;
    node->parent = NULL;
    node->children = NULL;
    node->next = NULL;

    return node;
}
```

src/tree.c — create_node(), in full.

Walking through it in the order the problem statement in Section “Learning Requirement” asks for:

- **Validation first.** `is_valid_name()` rejects empty names, names over 127 characters, the reserved names `.` and `..`, and any name containing `/` (which would let a single “name” smuggle in extra path components). Rejecting bad input before allocating anything means a failure here never leaves a half-built node that needs to be cleaned up.

- **Allocation.** `calloc(1, sizeof(Node))` asks for one zero-filled block exactly the size of a `Node`. Using `calloc` instead of `malloc` here is deliberate: it guarantees `parent`, `children`, `next`, and `content` all start as `NULL` without four separate assignment lines being relied upon to get it right (the explicit assignments below are kept anyway, as documentation of intent and a safety net).
- **Failure handling.** Every allocation is checked. If it fails, `create_node()` reports the problem and returns `NULL` rather than continuing with a null pointer that would eventually crash somewhere much harder to diagnose.
- **String copying.** `strncpy(node->name, name, MAX_NAME_LEN)` copies at most `MAX_NAME_LEN` bytes — but `strncpy` famously does *not* guarantee NUL-termination if the source is exactly that long, so the very next line, `node->name[MAX_NAME_LEN] = '\0';`, terminates it explicitly and unconditionally. This is defensive: `is_valid_name()` already guarantees the name fits, but the code does not rely on that guarantee holding forever as the codebase changes.
- **Type and pointer initialization.** The node's type is recorded, and every pointer field is set to a known, safe value (mostly redundant with `calloc`, again for clarity). The node is returned *detached* — it has no parent and is not part of any tree yet. That is `add_child()`'s job.

8. Adding Children

```
int add_child(Node *parent, Node *child)
{
    if (parent == NULL || child == NULL) return 0;
    if (parent->type != DIRECTORY_NODE) return 0;
    if (find_child(parent, child->name) != NULL) return 0; /* duplicate */

    child->parent = parent;
    child->next = NULL;

    if (parent->children == NULL) {
        parent->children = child;
        return 1;
    }

    Node *cursor = parent->children;
    while (cursor->next != NULL) {
        cursor = cursor->next;
    }
    cursor->next = child;
    return 1;
}
```

src/tree.c — add_child(), in full.

Three guard checks come first: neither pointer may be `NULL`, the parent must actually be a directory (files cannot have children), and the name must not already be taken inside that parent (this is where TreeFS enforces that every name in a directory is unique, regardless of whether the existing entry is a file or a directory). Only once all three pass does the function touch any pointers.

Linking then proceeds in two possible ways. If the parent currently has no children at all, the new node becomes `parent->children` directly. Otherwise, the function walks the existing sibling chain to its end (the last node whose `next` is `NULL`) and attaches the new node there. Appending at the end, rather than at the front, is what makes `ls`, `tree`, and the persistence layer's `save` order all agree with each other and with

creation order — a directory's entries always appear in the order you created them.

```
BEFORE add_child(projects, ml):

    projects
    |
    v
    dsa --> NULL

AFTER add_child(projects, ml):

    projects
    |
    v
    dsa --next--> ml --next--> NULL
```

Figure 8.1 — appending “ml” onto “projects”, whose only prior child was “dsa”.

9. Searching the Tree

```
Node *find_child(Node *parent, const char *name)
{
    if (parent == NULL || name == NULL) return NULL;

    for (Node *cursor = parent->children; cursor != NULL; cursor = cursor->next) {
        if (strcmp(cursor->name, name) == 0) {
            return cursor;
        }
    }
    return NULL;
}
```

src/tree.c — find_child(), in full.

`find_child()` only ever looks at the *direct* children of one node — it walks the sibling chain, comparing names, and stops the moment it finds a match (or runs out of siblings). This single-level search is exactly what `cd dsa` needs: “does the current directory have a child named ‘dsa’?” It is deliberately *not* recursive, because a single path component should never accidentally match something several levels deeper.

Finding every node named `tree.c` anywhere in the whole filesystem, which is what the `find` command does, is a different problem that needs to visit every node in the tree, not just one node's children. That is `find_in_tree()`, which is recursive and is covered together with recursion itself in the next section.

10. Recursion

A recursive function is one that calls itself on a smaller version of the same problem. Every correct recursive function needs two things: a **base case** — a version of the problem small enough to answer directly, with no further recursive call — and a **recursive case**, which does a small amount of work and then hands off a strictly smaller sub-problem to another call of itself. Without a reachable base case, the calls never stop, and the program eventually crashes when it runs out of **call stack** space — the region of memory where each in-progress call keeps its own local variables and “return to here when done” address, stacked on top of each other like plates.

Trees and recursion fit together almost perfectly, because “a tree” is itself defined recursively: a tree is a node, plus a collection of smaller trees (its children’s subtrees). A function that processes “this node, then every child’s subtree” is therefore following the data structure’s own shape.

Trace: `free_tree()` on a tiny example

```
free_tree(projects)
|
+-- free_tree(dsa)
|   |
|   +-- free_tree(stack.c)  <- no children: frees stack.c, returns
|   +-- free_tree(tree.c)   <- no children: frees tree.c, returns
|   (now free dsa itself)
|
+-- free_tree(README.txt)    <- no children: frees README.txt, returns
(now free projects itself)
```

Figure 10.1 — call order for `free_tree(projects)`. Every call’s own cleanup happens only after all of its children’s calls have returned — this is called “post-order” traversal.

The base case for `free_tree()` is a node with no children (`node->children == NULL`): the loop over children simply never executes, and the function goes straight to freeing the node’s own content and the node itself. The recursive case is any node with at least one child: for each child, call `free_tree()` again, and only *after* every child has been fully freed, free this node. Section 18 walks through why the child’s next pointer has to be saved *before* the recursive call, not after.

The same shape — “do something with this node, then recurse into each child” — also drives `print_tree()` (Section 17) and `find_in_tree()` (used by the `find` command, Section 9): both are pre-order instead of post-order (they act on a node *before* recursing into its children, rather than after), but the underlying pattern of “recurse once per child, using the tree’s own shape as the recursion’s shape” is identical.

11. Paths

TreeFS’s path resolver has to understand five things: the root (`/`), “stay here” (`.`), “go up” (`..`), absolute paths, and relative paths. Rather than writing a special case for each of these, TreeFS reduces all of them to one general algorithm: split the path into components separated by `/`, then walk the tree one component at a time.

```

static FsError resolve_from(Node *start, const char *path, Node **out)
{
    char buf[MAX_PATH_LEN];
    strncpy(buf, path, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';

    Node *current = start;
    char *saveptr = NULL;
    char *token = strtok_r(buf, "/", &saveptr);

    while (token != NULL) {
        if (strcmp(token, ".") == 0) {
            /* stay in place */
        } else if (strcmp(token, "..") == 0) {
            if (current->parent != NULL) current = current->parent;
        } else {
            if (current->type != DIRECTORY_NODE) return FS_ERR_NOT_DIR;
            Node *next = find_child(current, token);
            if (next == NULL) return FS_ERR_NOT_FOUND;
            current = next;
        }
        token = strtok_r(NULL, "/", &saveptr);
    }

    *out = current;
    return FS_OK;
}

```

src/filesystem.c — resolve_from(), the heart of path resolution.

`strtok_r()` is used specifically because it treats any run of consecutive `/` characters, and any leading or trailing `/`, as a single separator — so `/projects//dsa/` tokenizes into exactly `["projects", "dsa"]`, with no empty-string components to special-case. Whether the walk starts at the root or at the current directory is decided once, before this function is even called, purely by looking at the first character of the original path.

Trace: resolving `/projects/dsa/tree.c`

```

path[0] == '/' --> start = root

token 1: "projects" -> find_child(root, "projects")      -> current = projects
token 2: "dsa"      -> find_child(projects, "dsa")        -> current = dsa
token 3: "tree.c"   -> find_child(dsa, "tree.c")          -> current = tree.c

no more tokens -> *out = tree.c, return FS_OK

```

Figure 11.1 — each component triggers exactly one `find_child()` call on the result of the previous one.

If any step's `find_child()` returns `NULL`, resolution stops immediately with `FS_ERR_NOT_FOUND`. If a non-final component turns out to be a file rather than a directory (you cannot “descend into” a file), it stops with `FS_ERR_NOT_DIR`. Both are checked before consuming the next token, so the resolver never dereferences a pointer into something that was not actually a directory.

12. mkdir

Tracing `mkdir` projects end-to-end through every layer:

```

"mkdir projects"                                (raw line from the user)
|
v  tokenize()
argv = ["mkdir", "projects"]                     (commands.c)
|
v  cmd_mkdir() checks argc == 2, then calls:
fs_mkdir(fs, "projects")                         (filesystem.c)
|
v  fs_resolve_parent() splits "projects":
parent = fs->cwd, name = "projects"
|
v  find_child(parent, "projects") == NULL? (duplicate check)
|
v  create_node("projects", DIRECTORY_NODE) (tree.c)
|
v  add_child(parent, new_node)                 (tree.c)
|
v  FS_OK propagates all the way back up; cmd_mkdir prints nothing on success

```

Figure 12.1 — the full path input takes through every module for a single mkdir.

Every step after tokenization returns an explicit success/failure signal (`FsError` for the filesystem layer, a `Node*` or `NULL` for the tree layer) that the caller checks before proceeding. There is no step where a failure silently propagates as a null pointer dereference further up the chain — Section 25 discusses this error-handling discipline further.

13. touch

`touch` follows the identical path through `commands.c` and `fs_resolve_parent()` as `mkdir` — in fact both are implemented as thin wrappers around one shared helper, `create_entry()`, that takes the `NodeType` as a parameter:

```

static FsError create_entry(FileSystem *fs, const char *path, NodeType type)
{
    Node *parent;
    char name[MAX_NAME_LEN + 1];

    FsError err = fs_resolve_parent(fs, path, &parent, name, sizeof(name));
    if (err != FS_OK) return err;
    if (find_child(parent, name) != NULL) return FS_ERR_ALREADY_EXISTS;

    Node *node = create_node(name, type);
    if (node == NULL) return FS_ERR_MEMORY;
    if (!add_child(parent, node)) {
        free_tree(node);
        return FS_ERR_ALREADY_EXISTS;
    }
    return FS_OK;
}

FsError fs_mkdir(FileSystem *fs, const char *path) { return create_entry(fs, path, DIRECTORY_NODE); }
FsError fs_touch(FileSystem *fs, const char *path) { return create_entry(fs, path, FILE_NODE); }

```

`src/filesystem.c` — the only difference between `mkdir` and `touch` is one enum value.

The only real difference between a directory and a file, at creation time, is the `NodeType` passed in. A freshly-touched file's content is `NULL` (not an empty string) — TreeFS distinguishes “never written to” from

“explicitly written with empty content”, and both are tested for separately in the automated test suite (Section 26).

14. ls

```
void fs_ls(const FileSystem *fs)
{
    for (Node *child = fs->cwd->children; child != NULL; child = child->next) {
        printf("%s%s\n", child->name, child->type == DIRECTORY_NODE ? "/" : "");
    }
}
```

src/filesystem.c — fs_ls(), in full.

This is the simplest possible use of the sibling chain: start at `fs->cwd->children` (the current directory's first child), print its name, follow `next`, and repeat until `NULL`. No recursion is needed — unlike `tree`, `ls` only ever looks one level deep. Directory entries get a trailing `/` appended purely for readability, matching the convention used by `tree` (Section 17); it is not stored anywhere and has no effect on how the name is looked up.

15. cd

`fs_cd()` is almost entirely just `fs_resolve()`, with one extra check: whatever the path resolves to must actually be a directory (you cannot `cd` into a file). If so, `fs->cwd` is reassigned to point at it.

```

      root
      |
    projects
      |
      dsa    <-- fs->cwd currently points here

cd .. => fs->cwd = fs->cwd->parent => fs->cwd now points at "projects"
```

Figure 15.1 — `cd ..` is a single pointer reassignment, made possible entirely by the parent pointer every node already carries.

This is the payoff of storing a parent pointer on every node, even though parent is never needed while walking *downward* through a path: without it, moving up a level would require re-walking the entire tree from the root looking for “whoever has this node as a child”, which is exactly the kind of avoidable inefficiency a parent pointer exists to prevent.

16. pwd

TreeFS never stores a directory's absolute path as a string anywhere — doing so would mean updating potentially many stored strings every time something was renamed or moved. Instead, `pwd` reconstructs the path on demand, by walking *upward* from the current directory to the root, collecting each node's own (one-component) name along the way, and then printing those names in *reverse* order — because the walk naturally visits them leaf-to-root, but a path reads root-to-leaf.

```

char *fs_pwd(const FileSystem *fs, char *buf, size_t buf_size)
{
    const Node *chain[256];
    int depth = 0;
    const Node *cur = fs->cwd;

    while (cur != NULL && cur != fs->root) {
        if (depth < 256) chain[depth++] = cur;
        cur = cur->parent;
    }

    if (depth == 0) { snprintf(buf, buf_size, "/"); return buf; }

    size_t pos = 0;
    for (int i = depth - 1; i >= 0; i--) {
        int written = snprintf(buf + pos, buf_size - pos, "%s", chain[i]->name);
        pos += (size_t)written;
    }
    return buf;
}

```

src/filesystem.c — fs_pwd() (trimmed slightly for this listing; see the full version in the source for the bounds-checking details).

The chain array is filled leaf-to-root (chain[0] is the deepest directory, the last entry is the shallowest, just under the root), and then the printing loop walks it backwards (i from depth - 1 down to 0), which puts the shallowest component first — exactly the order a path is supposed to read in. If the current directory *is* the root, the while loop's condition is false on the very first check, depth stays 0, and the function takes the early “just print /” branch.

17. tree

```

void print_tree(const Node *node, const char *prefix)
{
    Node *child = node->children;
    while (child != NULL) {
        int is_last = (child->next == NULL);
        const char *connector = is_last ? "\u2514\u2500\u2500 " : "\u251C\u2500\u2500 ";

        printf("%s%s%s\n", prefix, connector, child->name,
            child->type == DIRECTORY_NODE ? "/" : "");

        if (child->type == DIRECTORY_NODE) {
            char child_prefix[4096];
            snprintf(child_prefix, sizeof(child_prefix), "%s%s", prefix,
                is_last ? " " : "\u2502 ");
            print_tree(child, child_prefix);
        }
        child = child->next;
    }
}

```

src/tree.c — print_tree(), in full (Unicode escapes shown for the box-drawing characters █, █, and █).

The recursive idea is: for every child of this node, print a line for it, and if that child is itself a directory, recursively print *its* children too — indented one level further. The interesting engineering detail is how the indentation is built: rather than tracking a numeric depth and multiplying by some fixed number of spaces, print_tree() passes down an already-assembled prefix string, and each recursive call extends it by

exactly the right connector for whether the current node was the *last* child at its level (a trailing directory gets plain spaces, so the vertical line does not stick out below where the branch actually ends) or not (it gets a vertical bar, ■, so following siblings' branches still connect visually).

```

/
■ ■ ■ projects/
  ■ ■ ■ dsa/
    ■ ■ ■ stack.c
    ■ ■ ■ tree.c
  ■ ■ ■ ml/
    ■ ■ ■ README.txt

```

Figure 17.1 — actual output of the ``tree`` command on a small filesystem. Note how the ■ under “dsa/” only appears because dsa was not the last child of projects/ — it stops as soon as the sibling list under projects/ is exhausted.

18. rm

Deleting a node safely means getting three things right, in order: unlink it from its parent's sibling list so nothing can find it anymore, free everything it owns (including, recursively, everything beneath it), and never leave any *other* live pointer — especially `fs->cwd` — referring to memory that was just freed.

Design decision: rm refuses non-empty directories

TreeFS deliberately does **not** support recursive deletion of a non-empty directory. `fs_rm()` checks `target->children != NULL` and, if true, refuses with `FS_ERR_NOT_EMPTY` rather than silently deleting everything inside. The alternative — walking the subtree and freeing it all — is not meaningfully harder to implement (it is a two-line change: call `free_tree()` instead of requiring emptiness first), but it is meaningfully more dangerous to use: a single mistyped `rm` on the wrong directory could silently erase an entire project tree with no confirmation prompt and no undo. Requiring the directory to be empty first turns any accidental large-scale deletion into a sequence of individually-visible, individually-reversible steps. This mirrors the classic Unix distinction between plain `rmdir` (empty only) and `rm -r` (recursive, opt-in, and famous for its capacity to cause damage when used carelessly).

Refusing to remove / or the current directory

```

FsError fs_rm(FileSystem *fs, const char *path)
{
    Node *target;
    FsError err = fs_resolve(fs, path, &target);
    if (err != FS_OK) return err;
    if (target == fs->root) return FS_ERR_ROOT;
    if (target->type == DIRECTORY_NODE && target->children != NULL) return FS_ERR_NOT_EMPTY;
    if (target == fs->cwd) return FS_ERR_IS_CWD;

    if (!remove_child(target->parent, target)) return FS_ERR_NOT_FOUND;
    free_tree(target);
    return FS_OK;
}

```

src/filesystem.c — fs_rm(), in full.

The `target == fs->cwd` check exists for a subtle reason: if you are currently sitting inside an *empty* directory and remove it by an absolute path from where you stand, the empty-directory check alone would let

that through — and `fs->cwd` would then be a dangling pointer into freed memory, which the very next `pwd` or `ls` would read from. Refusing this case outright is far simpler and safer than trying to detect and repair a dangling `cwd` after the fact.

Unlinking, in `tree.c`

```
int remove_child(Node *parent, Node *child)
{
    if (parent->children == child) {
        parent->children = child->next;
        child->next = NULL;
        child->parent = NULL;
        return 1;
    }
    Node *cursor = parent->children;
    while (cursor != NULL && cursor->next != child) cursor = cursor->next;
    if (cursor == NULL) return 0;
    cursor->next = child->next;
    child->next = NULL;
    child->parent = NULL;
    return 1;
}
```

src/tree.c — `remove_child()`, in full.

Two cases: the node being removed is the *first* child (its parent's `children` pointer must be updated directly to skip over it), or it is somewhere later in the chain (the *preceding* sibling's `next` is updated instead, since singly-linked lists can only be unlinked by adjusting the link that points *into* the node being removed). Either way, the removed node's own `next` and `parent` are reset to `NULL` — it is now a fully detached, self-contained subtree, safe to pass to `free_tree()` without that call accidentally wandering back into the live tree through a stale pointer.

19. File Contents

A file's content in TreeFS is nothing more than an ordinary heap-allocated, NUL-terminated `char *`, sized to fit exactly what was written — never a fixed-size buffer, and never pre-allocated before it is needed.

```
FsError fs_write(FileSystem *fs, const char *path, const char *content)
{
    Node *target;
    FsError err = fs_resolve(fs, path, &target);
    if (err != FS_OK) return err;
    if (target->type != FILE_NODE) return FS_ERR_IS_DIR;

    size_t len = strlen(content != NULL ? content : "");
    char *copy = malloc(len + 1);
    if (copy == NULL) return FS_ERR_MEMORY;
    memcpy(copy, content != NULL ? content : "", len + 1);

    free(target->content);    /* release whatever was there before */
    target->content = copy;
    return FS_OK;
}
```

src/filesystem.c — `fs_write()`, in full.

Notice the order of the last three lines: a brand-new buffer is allocated and filled *first*, and only once that has fully succeeded is the *old* content buffer freed and replaced. If the `malloc()` for the new content had failed, returning early at that point leaves `target->content` completely untouched — a failed write never destroys existing data. A brand-new file straight out of touch has `content == NULL`: it exists but has never been written to, which is exactly why `cat` on an untouched file prints nothing rather than an error, matching real-shell behavior on an empty file.

`write` always *replaces* a file's entire content rather than appending to it — there is no separate append mode in this project, by design, to keep the command's behavior unambiguous: one call, one complete new value.

20. Persistence

Everything built up through Section 19 lives entirely inside one running process's heap memory, which is **volatile**: the instant the `treefs` process exits — normally via `exit`, or the terminal closing, or the machine losing power — every `Node` and every byte of file content disappears with it. Without persistence, TreeFS would only ever exist for the length of one program run, which the project brief explicitly requires it not to be.

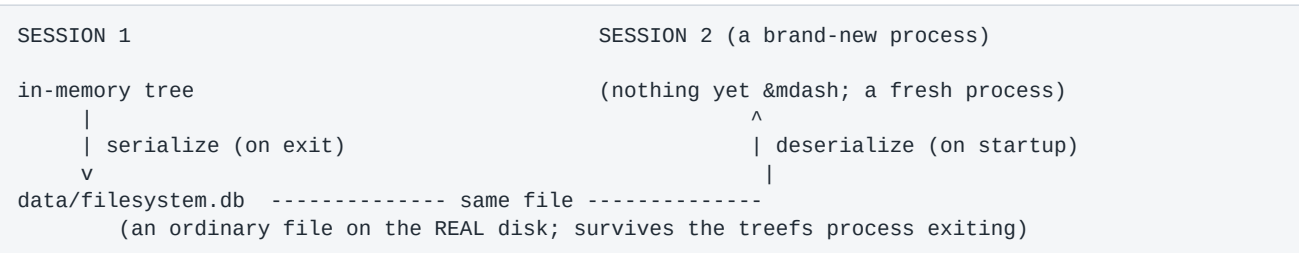


Figure 20.1 — the database file is the only thing that bridges two separate process lifetimes.

The fix is the standard one: before the tree disappears, write ("serialize") enough information about it to an ordinary file that a fresh process can later read that file and rebuild ("deserialize") an identical tree. `main.c` calls `save_filesystem()` unconditionally right before it returns, and calls `load_filesystem()` exactly once, at startup, before the first prompt is ever shown.

21. The Serialization Format

TreeFS uses a flat, length-prefixed text format. Every node in the tree, except the implicit root (which every fresh filesystem already has), becomes one record of exactly this shape:

```

<TYPE> <PATH_LEN> <CONTENT_LEN>\n
<PATH bytes, exactly PATH_LEN of them>\n
<CONTENT bytes, exactly CONTENT_LEN of them>\n

```

`TYPE` is a single character, `D` (directory) or `F` (file). `PATH_LEN` and `CONTENT_LEN` are decimal byte counts. Below is an actual database file produced by a real TreeFS session — this exact file was generated and then read back during the verification work for this guide (Section 26):

```

TREEFS_V1
D 9 0
/projects

D 13 0
/projects/dsa

F 21 0
/projects/dsa/stack.c

F 20 0
/projects/dsa/tree.c

D 12 0
/projects/ml

F 20 20
/projects/README.txt
hello from session 1

```

An actual data/filesystem.db written by the compiled binary.

The full *absolute path* of every node is stored, not just its bare name — this is what makes each record self-describing: the loader never has to infer where a node belongs relative to the previous record, because every record states its own complete location outright. Records are written in **pre-order** (a node is always written before anything inside it), which falls out naturally from how `write_subtree()` walks the tree, and is exactly the order the loader needs, since a directory must exist before anything can be created inside it.

Why length-prefixing instead of one-item-per-line text

The tempting simpler format is one path (or one content blob) per line, split on newline characters. That format breaks the moment file content itself contains a newline: `write notes.txt "line one\nline two"` would corrupt the layout, because a naive parser could not distinguish "this newline ends the record" from "this newline is part of the data". Prefixing every path and every content blob with its exact byte length sidesteps the problem entirely — `fread()` is told precisely how many bytes to consume, so embedded newlines, embedded quote characters, or any other byte value are all handled correctly with zero escaping logic. The trailing `\n` after each field exists purely for human readability when eyeballing the file by hand; the parser itself never depends on it to know where a field ends.

22. Deserialization

`load_filesystem()` reads the header line, then loops, reading one record at a time: `fscanf()` reads the type character and the two lengths, an exact-byte `fread()` pulls in the path, another exact-byte `fread()` pulls in the content (if any), and the record is then **replayed** through the very same operations the interactive shell itself calls:

```

FsError err;
if (type == 'D') {
    err = fs_mkdir(fs, path);
} else {
    err = fs_touch(fs, path);
    if (err == FS_OK && content_len > 0) err = fs_write(fs, path, content);
}

```

Replaying through the ordinary operations, rather than constructing Node structs directly, is a deliberate design choice: it makes the loader structurally incapable of producing a tree that violates any invariant the rest of the program already enforces — whatever `fs_mkdir()` would refuse from a human typing at the prompt, it also refuses when the same path shows up in a database file.

Handling a malformed file without crashing

Every field read from disk is validated before use: the type must be D or F; `path_len` must be nonzero and under `MAX_PATH_LEN`; `content_len` is capped at 16 MiB (`MAX_RECORD_CONTENT_LEN`) specifically so a corrupted or hand-edited length field cannot trigger an enormous allocation; and every expected newline is checked with a small `expect_newline()` helper. Any violation returns `PERSIST_ERR_MALFORMED` immediately, with no attempt to salvage partial data. Critically, `main.c` loads into a *separate, temporary* Filesystem first and only swaps it into place if loading fully succeeds — so a corrupted database file can never leave the live, in-use filesystem half-populated; the program instead falls back to an empty filesystem and prints a warning.

23. Command Parsing

A line such as `write hello.txt "Hello World"` must become three separate tokens: `write`, `hello.txt`, and `Hello World` as a single token, with the quote characters stripped but the interior space preserved. `tokenize()` handles this with one left-to-right scan: runs of plain whitespace separate ordinary tokens, but a token that *begins* with a double quote is instead read up to its matching closing quote.

```
if (*p == '"') {
    p++;
    argv[argc++] = p;
    char *write_ptr = p;
    while (*p != '"' && *p != '\0') { *write_ptr++ = *p++; }
    if (*p == '\0') { fprintf(stderr, "unterminated quoted string\n"); return -1; }
    *write_ptr = '\0';
    p++;
}
/* consume closing quote */
```

src/commands.c — the quoted-token branch of tokenize().

Tokens are produced **in place**, inside the very same buffer the raw input line was read into — every `argv[i]` points directly into that buffer rather than into a freshly allocated string. This is exactly why `tokenize()` needs no matching "free the tokens" step afterward: their lifetime is simply the lifetime of the one input-line buffer declared in `main()`'s loop, which is naturally overwritten on the next iteration anyway. An unterminated quote, or more tokens than `MAX_ARGS` (8) allows, is reported immediately and the whole line is discarded rather than partially executed.

24. Modular Architecture

File	Owns	Knows nothing about
tree.c / tree.h	The Node struct and pure tree operations: create, link, unlink, free, print, search	Paths, the current working directory, error codes, or commands
filesystem.c / .h	The FileSystem struct (root + cwd), path resolution, and every fs_* operation	Command-line syntax, quoting, or stdin
commands.c / .h	Tokenizing a raw input line and dispatching it to the right fs_* call	How a Node is represented internally
persistence.c / .h	The on-disk record format; reads/writes by calling fs_* functions only	Command-line syntax
main.c	The read-execute loop, plus load-at-startup and save-at-exit	Everything above — it only ever calls into the other four layers

This split follows one rule throughout: **each layer calls only downward, never sideways or upward**. tree.c has no `#include "filesystem.h"` anywhere in it, so a future change to path syntax can never require touching the tree layer at all. The practical payoff is that each file can be understood, and tested, in isolation — exactly what the automated test suite (Section 26) does, linking tree.c and filesystem.c together without ever pulling in commands.c or main.c.

25. Memory Management

Object	Allocated by	Freed by
A Node	create_node() (the root is a special case, built directly in fs_init())	free_tree() (recursively) or fs_destroy() at program exit
Node.content buffer	fs_write() — malloc'd exactly to the size of the new content	fs_write() itself frees the old buffer before replacing it; free_tree() frees the final buffer on deletion
Path-resolution scratch buffers (fs_resolve, fs_pwd, print_tree prefixes)	Fixed-size stack arrays local to each function	Automatically, when the function returns — never heap, never freed explicitly
Command tokens (argv[])	Not allocated — point directly into the existing line[] buffer	Not freed — the buffer is simply reused on the next input line
Deserialized content during load	load_filesystem() (malloc, one buffer per record with content)	load_filesystem(), immediately after each record is replayed

Every Node has exactly one owner at any given moment: either it is reachable from `fs->root` (owned by the live tree), or it is a freshly created, not-yet-attached node owned solely by whichever function currently holds the only pointer to it. `create_entry()` shows the detached case directly: if `add_child()` ever fails

immediately after `create_node()` already succeeded, the orphaned node is explicitly passed to `free_tree()` rather than left to leak.

The four classic C memory bugs, and how each is avoided here

- **Memory leak** — every `Node` and every content buffer has exactly one of two fates: freed by `free_tree()` on removal or program exit, or freed immediately by the function that allocated it if attaching it fails. Verified empirically with Valgrind (Section 26): 0 bytes leaked across every run performed.
- **Double free** — avoided because ownership is single and explicit: once `remove_child()` unlinks a node, nothing else in the program retains a pointer to it except the one local variable about to be passed to `free_tree()` exactly once.
- **Use-after-free** — avoided by the "save the next pointer before recursing" pattern used throughout `free_tree()` and `print_tree()`, and by always unlinking a node from its parent *before* freeing it in `fs_rm()`, never after.
- **Dangling pointer** — guarded structurally: `remove_child()` resets both `child->next` and `child->parent` to `NULL` the instant a node is unlinked, and `fs->cwd` can never end up pointing at freed memory because `fs_rm()` explicitly refuses to remove the current directory.

26. Testing: What Was Actually Run

This section reports only checks that were actually executed against the compiled binary while preparing this guide — nothing here is a prediction of what the tests would probably do.

26.1 Build

```
$ make
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c -o build/main.o src/main.c
... (four more files) ...
gcc -std=c11 -Wall -Wextra -Wpedantic -g -o treefs build/*.o

Result: exit code 0, zero compiler warnings, zero errors.
```

26.2 Automated test suite (tests/test_treefs.c)

Built and linked directly against `tree.c`, `filesystem.c`, and `persistence.c` — bypassing the interactive shell entirely — and run with `make test`:

```

test_root_creation
test_directory_and_file_creation
test_duplicate_creation
test_absolute_and_relative_navigation
test_parent_navigation
test_file_write_and_read
test_deletion
test_rename
test_find_recursive
test_invalid_paths_and_names
test_recursive_tree_traversal
test_persistence_round_trip
test_persistence_missing_and_malformed_file

```

82/82 checks passed

26.3 Interactive end-to-end session, including a real process restart

The exact session from the project brief was typed into a real, running `./treefs` process via stdin redirection, ended with `exit` (a genuine save to `data/filesystem.db`), and then a **separate, freshly-started** process was launched against that same file:

```

SESSION 1 (abbreviated)
treefs:/ $ mkdir projects
treefs:/ $ cd projects
treefs:/projects $ mkdir dsa
treefs:/projects $ mkdir ml
treefs:/projects $ touch README.txt
treefs:/projects $ cd dsa
treefs:/projects/dsa $ touch stack.c
treefs:/projects/dsa $ touch tree.c
treefs:/projects/dsa $ ls
stack.c
tree.c
treefs:/projects/dsa $ cd ..
treefs:/projects $ write /projects/README.txt "hello from session 1"
treefs:/projects $ exit

--- process exited; a brand new "./treefs" process was then started ---

SESSION 2 (new process, same data/filesystem.db)
treefs:/ $ tree
/
projects/
  dsa/
    stack.c
    tree.c
  ml/
  README.txt
treefs:/ $ cat /projects/README.txt
hello from session 1

```

The tree and the file content survived the restart exactly as required by the project brief. This confirms persistence works end to end, not merely that `save_filesystem()` and `load_filesystem()` pass in isolation inside the unit tests.

26.4 Invalid-input / error-handling session

Every invalid case listed in the project brief was exercised in one live session against the running shell: an existing name reused for mkdir/touch, cd into a nonexistent path, cd into a file, rm/cat/write on a nonexistent file, cat/write on a directory, rename of a nonexistent file, rm of a non-empty directory, an unrecognized command, and missing arguments. Every single case produced a clear treefs: <command>: <reason> message on stderr, and the shell kept running afterward; the process's final exit code was 0 and the prompt never stopped responding.

26.5 Memory safety: Valgrind

Valgrind 3.22 was available in the build environment and was run with full leak checking, both on the interactive session above and on the automated test binary:

```
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./treefs < session.txt
==863== HEAP SUMMARY:
==863==    in use at exit: 0 bytes in 0 blocks
==863==    total heap usage: 16 allocs, 16 frees, 14,851 bytes allocated
==863== All heap blocks were freed -- no leaks are possible
==863== ERROR SUMMARY: 0 errors from 0 contexts

$ valgrind --leak-check=full ./build/test_treefs
==869==    in use at exit: 0 bytes in 0 blocks
==869==    total heap usage: 71 allocs, 71 frees, 35,855 bytes allocated
==869== All heap blocks were freed -- no leaks are possible
==869== ERROR SUMMARY: 0 errors from 0 contexts
```

Both runs report zero leaked bytes and zero errors of any kind — no invalid reads or writes, no use of uninitialized values, no invalid frees. No test in this section was skipped, simulated, or approximated; Valgrind was available in the environment and used directly against the real compiled binary.

27. Complexity Analysis

Let k be the number of entries in one directory (the length of one sibling chain), d be the depth of a path (its number of components), and n be the total number of nodes in the whole tree. Every bound below follows directly from the child/sibling representation introduced in Section 4.

Operation	Time	Why
find_child (one level)	$O(k)$	Linear scan of one sibling chain
add_child	$O(k)$	Walks to the end of the sibling chain to append
remove_child	$O(k)$	Linear scan to find the node's preceding sibling
fs_resolve (path lookup)	$O(d \times k_{\text{avg}})$	One find_child call per path component
ls	$O(k)$	One pass over the current directory's sibling chain
pwd	$O(d)$	One parent-pointer step per ancestor, walked once
tree (full print)	$O(n)$	Visits every node exactly once

Operation	Time	Why
find (recursive search)	$O(n)$	Visits every node once; reports all matches, so no early exit
free_tree	$O(n)$	Visits every node exactly once, in post-order
save_filesystem	$O(n + \text{total content bytes})$	One record written per node; content copied once
load_filesystem	$O(n \times k_{\text{avg}} + \text{content bytes})$	Each record replayed through fs_mkdir/fs_touch, each itself $O(k)$

The recurring $O(k)$ cost of a linear sibling-chain scan is the direct price of the design chosen in Section 4: an array-of-children representation would offer $O(1)$ indexed access but $O(k)$ removal (to shift elements down) plus extra per-node capacity bookkeeping; the sibling-list representation instead makes every mutation $O(k)$ and every Node exactly two pointers wide regardless of how many children it has. That is the more favorable trade for directories holding the handful-to-hundreds of entries this project targets — not for directories with millions.

28. Design Decisions and Alternatives Considered

Decision	Alternative considered	Why this project chose as it did
N-ary tree with parent pointers	A flat hash map of full-path to node	A hash map loses the hierarchy needed for free traversal (tree, find, ..) and needs manual path-prefix bookkeeping to fake it back in; a tree gives "walk to parent" and "walk the subtree" as $O(1)$ pointer follows and $O(n)$ recursion respectively, with no extra structure required.
First-child / next-sibling	A fixed array of child pointers, grown with realloc	The sibling-list keeps every Node a constant size and makes append/remove ordinary linked-list operations rather than array-shift operations (Section 4).
Text, length-prefixed serialization	A raw binary struct dump of each Node	A struct dump would embed pointer values meaningless in a new process and embed compiler padding bytes, and be unrecoverable by hand; the format in Section 21 is inspectable in a plain text editor and immune to embedded-newline corruption.
Refuse rm on a non-empty directory	Recursive delete of the whole subtree	See the safety rationale in Section 18 — a single mistake should never be able to destroy more than the one name that was typed.
User-space, in-memory tree plus serialization	A real FUSE filesystem mounted into the OS	FUSE would require kernel/OS integration and platform-specific build steps that teach little about the core data structures and algorithms this project exists to teach; Section 29 details what a real filesystem does on top of this.
Interactive REPL shell	A single-shot CLI tool (one command per process invocation)	A REPL keeps the tree resident across many commands in one session, which is what makes a stateful current working directory meaningful at all.

29. What a Real File System Does Differently

TreeFS intentionally leaves out everything below. Naming what is missing is itself part of understanding what a real filesystem is for.

- **Blocks and disk sectors** — real filesystems store data in fixed-size chunks (commonly 4 KiB blocks) placed at specific physical locations on a storage device; TreeFS content is one contiguous heap buffer per file, with no concept of a "block" at all.
- **Inodes** — a real file's metadata (size, permissions, timestamps, block pointers) is stored separately from its directory entry, which is what lets the same content be referenced by multiple names (hard links). TreeFS keeps no separate metadata record: name and content live together in one Node.
- **Free-space tracking and block allocation** — a real filesystem must track which disk blocks are currently unused; TreeFS simply asks the C heap allocator for memory and has no notion of "free space" bookkeeping of its own.
- **Journaling** — real filesystems log pending changes before applying them so a power loss mid-write cannot corrupt the structure; TreeFS's persistence is an all-or-nothing full save performed only on exit, with no protection if the process is killed mid-write.
- **Permissions and ownership** — real filesystems track a user/group owner and read/write/execute bits per file; every TreeFS session has unrestricted access to everything in the tree.
- **File descriptors and caching** — a real OS lets many processes hold independent open handles (with their own read/write offsets) to the same file, and caches recently used blocks in RAM; TreeFS has no file-handle concept — every operation resolves the path fresh, every time.
- **Concurrency** — a real filesystem must handle multiple processes reading and writing at once without corrupting shared structures; TreeFS is single-threaded and assumes exactly one shell session owns the data file at a time.

30. Realistic Future Extensions

None of the following exist in the current implementation; they are documented here as directions a student could take the project next, not as claims about current behavior.

- **Metadata** — creation/modification timestamps and a stored file size, as extra Node fields and extra fields in the persistence record.
- **Permissions** — a simple owner-plus-rwx-bits field, checked at the start of each `fs_*` operation.
- **Hard links / symbolic links** — would require separating "directory entry" from "file content" (moving toward an inode-like layer), since a hard link is two names sharing one underlying content object.
- **Binary serialization** — a fixed-layout binary record format in place of text, trading human-readability for a smaller file and a faster load.
- **An inode abstraction** — separating metadata/content ownership from tree-position (name, parent, siblings), the real structural step toward hard-link support.
- **Block-based storage** — splitting file content into fixed-size chunks with a free-list, closer to how a real filesystem manages a physical disk.

- **A write-ahead journal** — logging each mutating operation before applying it, so a crash mid-session could be recovered from rather than only being safe at a clean exit.
- **Concurrency control** — a lock, or a single-writer/multiple-reader scheme, around the `FileSystem` struct if multiple threads or processes were ever allowed to touch the same tree.
- **FUSE integration** — wrapping the existing `fs_*` functions behind `libfuse` callbacks so the tree could actually be mounted and browsed with real OS tools — the cleanest possible next project, since `filesystem.c`'s API was deliberately kept OS-agnostic from the start.

31. Complete Code Walkthrough: How the Files Connect

This final section is a map, not a repetition of earlier sections — use it to know where to look once the source itself is open.

31.1 `tree.h / tree.c` (207 lines)

The lowest layer. Defines `Node` and `NodeType`, and seven functions that manipulate pointers only within a subtree: `is_valid_name`, `create_node`, `add_child`, `find_child`, `remove_child`, `free_tree`, `print_tree`, and `find_in_tree`. Nothing in this file knows what a "current directory" or a "path string" even is.

31.2 `filesystem.h / filesystem.c` (406 lines)

Owns the `FileSystem` struct and the `FsError` enum. Contains path resolution (`fs_resolve`, `fs_resolve_parent`) and every user-facing operation (`mkdir`, `touch`, `rm`, `cat`, `write`, `rename`, `cd`, `pwd`, `ls`, `find`), each implemented purely in terms of the tree-layer functions above it plus the resolver.

31.3 `commands.h / commands.c` (274 lines)

Owns `tokenize()` and the `COMMANDS[]` dispatch table mapping a command word to one handler function. Every `cmd_*` handler is a thin wrapper: check the argument count, call the matching `fs_*` function, and translate a non-OK `FsError` into a printed message via `fs_error_message()`.

31.4 `persistence.h / persistence.c` (210 lines)

Owns the on-disk record format from Section 21. `save_filesystem()` walks the live tree with `write_subtree()`; `load_filesystem()` parses records and replays them through the ordinary `fs_*` operations — it never constructs a `Node` directly.

31.5 `main.c` (99 lines)

The entry point, and nothing else: initializes a `FileSystem`, attempts to load `data/filesystem.db` into a separate temporary filesystem and only adopts it on full success, runs the prompt/read/tokenize/dispatch loop until `should_exit` is set, and then unconditionally saves and calls `fs_destroy()`.

31.6 tests/test_treefs.c (413 lines)

A self-contained, assertion-based test file compiled directly against `tree.c`, `filesystem.c`, and `persistence.c` — never against `commands.c` or `main.c`, since testing the logic does not need an interactive shell. Thirteen test functions cover creation, duplicates, navigation (absolute, relative, parent), reading/writing, deletion, renaming, recursive find, invalid-input handling, full-tree traversal, and a complete save-then-load round trip — 82 individual checks in total, all currently passing (Section 26).

This guide was written against, and verified against, the exact source in the accompanying project: every function quoted above is taken directly from the compiled, tested implementation, not reconstructed from the specification alone.